

ПРАКТИЧЕСКАЯ РАБОТА

Веб-приложение, доступное только через WireGuard VPN, с авторизацией OAuth 2.0

Отчёт по практической работе

Выполнил: Абянов Дамир Шагидинович

Группа: _____

Проверил: _____

Дата: _____

Москва, 2026

СОДЕРЖАНИЕ

1. Введение и цель работы	3
2. Как устроено веб-приложение	4
3. Настройка работы VPN	7
4. Интеграция авторизации OAuth 2.0	10
5. Выводы и сложности	14
Приложение А. Команды развёртывания, запуска и тестирования	15

1. Введение и цель работы

Цель и ожидаемый результат работы

Цель работы - создать многостраничное веб-приложение, закрыть сетевой доступ к нему туннелем WireGuard и встроить авторизацию по стандарту OAuth 2.0.

Проект решает две независимые задачи безопасности: сетевой уровень допускает соединение только с адреса VPN-клиента, а прикладной уровень проверяет личность пользователя перед открытием защищённой страницы.

Состав стенда

Компонент	Реализация	Состояние
Веб-приложение	Python 3, стандартная библиотека HTTP; маршруты /, /about, /private	Проверено локально
VPN	WireGuard, сеть 10.20.0.0/24, сервер 10.20.0.1	Запущено на 2 VM
Firewall	nftables: 8080/9000 только через wg0	Проверено curl
Авторизация	OAuth 2.0 Authorization Code + PKCE S256	Проверено с client VM

Фактический стенд

VirtualBox VM wg-server-damir (пользователь damir1) и wg-client-damir (пользователь damir2).
WireGuard, firewall, веб-служба и OAuth-сервер запущены; проверки выполнены с клиентской VM.

Репозиторий веб-приложения

<https://github.com/AbyanovDamir/vpn-project-github-with-vm-commands>
Подробная инструкция: [README.md](#)

2. Как устроено веб-приложение

Архитектура, маршруты и встроенные меры защиты

Приложение реализовано без внешних Python-зависимостей. Это упрощает развёртывание на чистой Ubuntu VM: достаточно Python 3 и systemd.

Маршруты:

URL	Назначение	Доступ
/	Главная страница и статус сервиса	Публичный внутри VPN
/about	Описание архитектуры	Публичный внутри VPN
/private	Профиль пользователя	После OAuth 2.0
/login, /callback	Запуск и завершение OAuth-потока	Служебные

Сессионные данные подписываются HMAC-SHA256 и хранятся в cookie с атрибутами HttpOnly и SameSite=Lax. На всех HTML-ответах выставляются X-Content-Type-Options и Content-Security-Policy.

Проверка

```
$ python scripts/verify_project.py
OK: Python syntax, 3 pages, OAuth PKCE and wg0-only firewall rule verified
```

```
$ python scripts/integration_oauth_test.py
OK: home, about, Authorization Code + PKCE, token exchange and private page
```

2.1. Страницы приложения

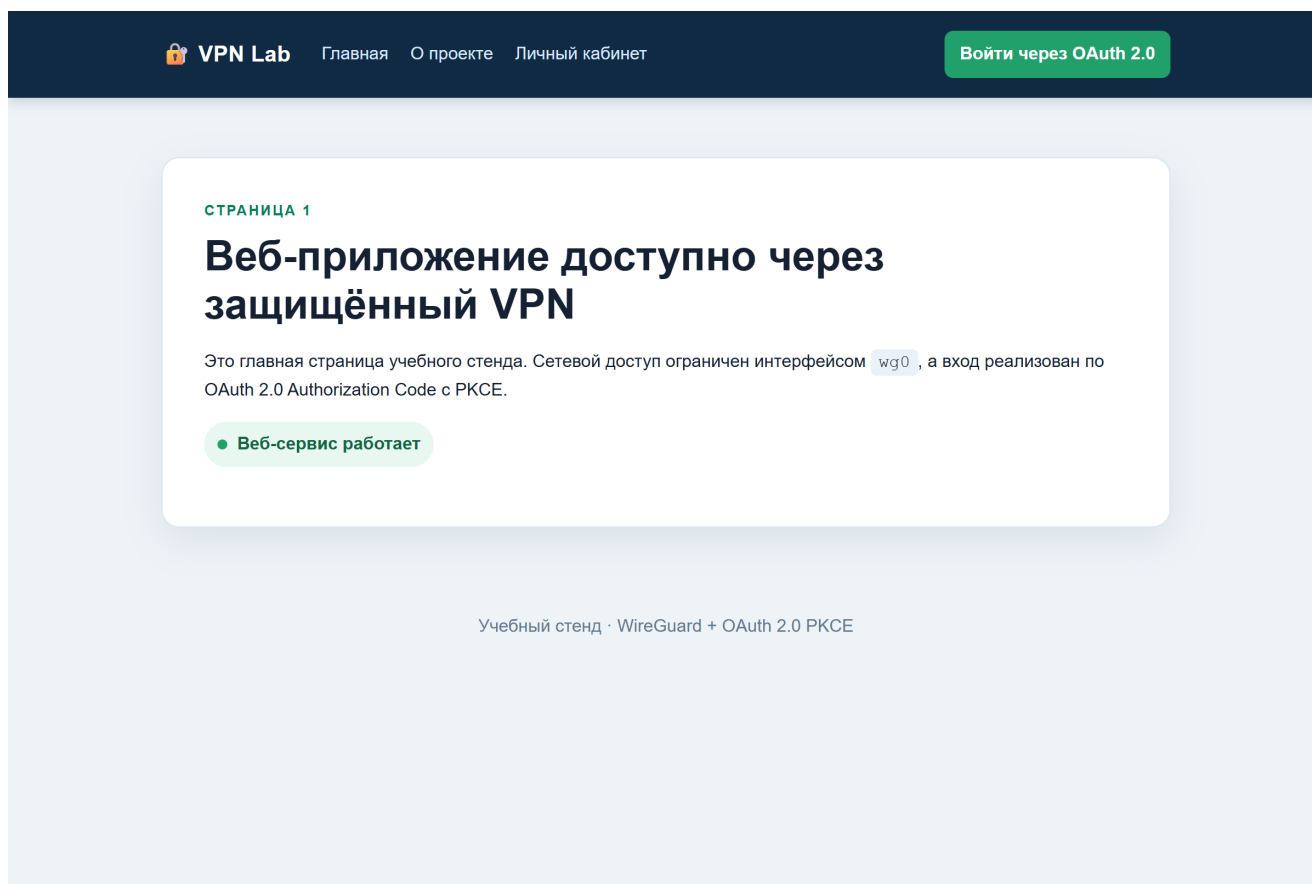


Рисунок 1 - главная страница приложения в desktop-браузере

СТРАНИЦА 2

О проекте

Стенд состоит из двух независимых HTTP-служб: клиентского веб-приложения и сервера авторизации. WireGuard создаёт частную сеть `10.20.0.0/24`.

Цель

Показать совместную работу прикладной авторизации и сетевого ограничения доступа.

Учебный стенд · WireGuard + OAuth 2.0 PKCE

Рисунок 2 - вторая страница «О проекте»

3. Настройка работы VPN

WireGuard между двумя Ubuntu VM и ограничение доступа через nftables

Выбран протокол WireGuard. Сервер wg-server-damir получает адрес 10.20.0.1/24, клиент wg-client-damir - 10.20.0.2/24. Через туннель маршрутизируется только приватная сеть 10.20.0.0/24, поэтому обычный интернет-трафик клиента не меняется.



Без активного VPN веб-приложение недоступно; nftables принимает HTTP только с wg0.

Схема 1 - сетевой путь к веб-приложению

Ключевые параметры

```
# server: /etc/wireguard/wg0.conf
[Interface]
Address = 10.20.0.1/24
ListenPort = 51820
PrivateKey = <server-private-key>
```

```
[Peer]
PublicKey = <client-public-key>
AllowedIPs = 10.20.0.2/32
```

```
# client
AllowedIPs = 10.20.0.0/24
PersistentKeepalive = 25
```

Доступ к TCP 8080 и 9000 разрешён только при входе пакета через интерфейс wg0. Политика input - drop. На публичном интерфейсе остаются только SSH и UDP 51820 для установления туннеля.

3.1. Серверная VM

Скриншот получен напрямую из консоли VirtualBox после развёртывания. В верхней части видны имя машины и пользователь; ниже - активные службы, VPN-адрес, слушающие сокеты и свежий WireGuard handshake.

```
VM:  wg-server-damir
USER: damir1

=== SERVER: WireGuard + web + OAuth ===
hostname: wg-server-damir
whoami:  damir1

wg-quick@wg0: active
vpn-web:  active
oauth-server: active
nftables: active

wg0      UNKNOWN      10.20.0.1/24
LISTEN 0      5              10.20.0.1:9000  0.0.0.0:*
LISTEN 0      5              10.20.0.1:8080  0.0.0.0:*

Interface: wg0
  public key: EyQJZyyK7vDMdVoIXNEHe9rFkeRrbK9Kgv9rIT10LTk=
  private key: (hidden)
  listening port: 51820

peer: D0QdRQVw16b/ZAHQYaNDE0N1B8UsakLxybJrQfMRKBS=
  endpoint: 192.168.100.20:35684
  allowed ips: 10.20.0.2/32
  latest handshake: 9 seconds ago
  transfer: 50.07 KiB received, 108.28 KiB sent
```

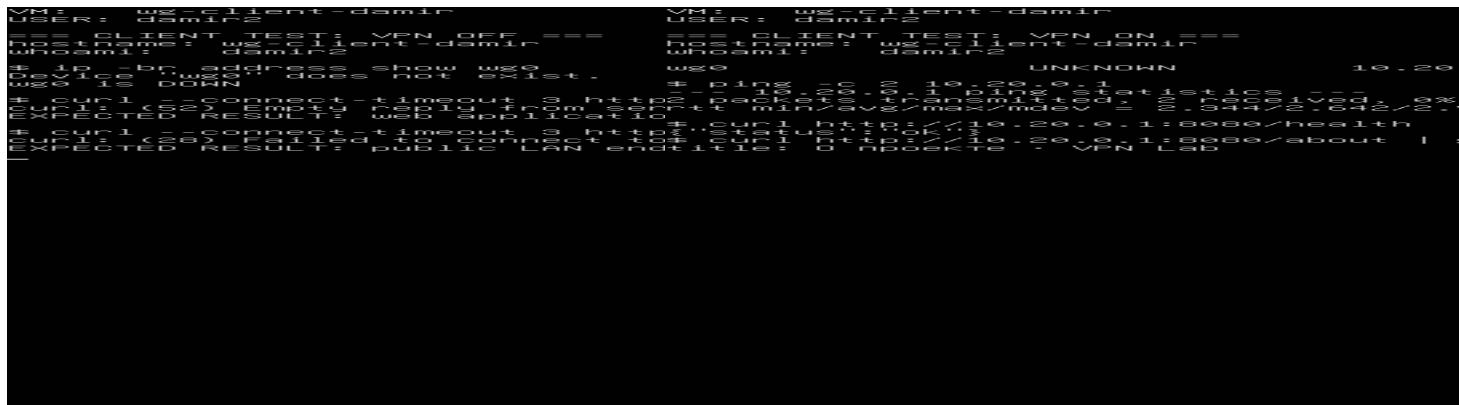
Рисунок 3 - wg-server-damir / damir1: WireGuard, nftables, web и OAuth активны

Сетевое ограничение

vpn-web и oauth-server слушают только 10.20.0.1, а nftables разрешает TCP 8080/9000 только на входе через wg0. На LAN-адресе 192.168.100.10 эти порты закрыты.

3.2. Проверка доступа с клиентской VM

Обе проверки выполнены на wg-client-damir. Слева интерфейс wg0 выключен: обращение к VPN-адресу не возвращает HTTP-ответ, а прямой LAN-адрес сервера завершается тайм-аутом. Справа после wg-quick up виден адрес 10.20.0.2/24, ping без потерь и ответы приложения.



Рисунки 4-5 - damir2@wg-client-damir: приложение недоступно без VPN и доступно после включения wg0

```
# VPN выключен
sudo wg-quick down wg0
curl --connect-timeout 3 http://10.20.0.1:8080/health # нет HTTP-ответа
curl --connect-timeout 3 http://192.168.100.10:8080/health # timeout

# VPN включен
sudo wg-quick up wg0
ping -c 2 10.20.0.1 # 0% packet loss
curl http://10.20.0.1:8080/health # {"status": "ok"}
```

Результат

Критерий доступа только с включённым VPN подтверждён фактическими проверками на клиентской VM.

4. Интеграция авторизации OAuth 2.0

Authorization Code с расширением PKCE S256

Использован поток Authorization Code с расширением PKCE S256. Сервер авторизации запущен отдельным процессом на порту 9000; клиентское приложение не получает пароль пользователя и работает только с одноразовым code и bearer-токеном.



state защищает от CSRF; PKCE связывает code с одноразовым code_verifier.

Схема 2 - последовательность OAuth 2.0 Authorization Code + PKCE

Контрольные меры

Механизм	Что проверяется
state	Совпадение значения в запросе и callback; защита от CSRF
PKCE S256	SHA-256 code_verifier совпадает с code_challenge
redirect_uri	Точное совпадение с зарегистрированным URI
Одноразовый code	Удаляется при первом обращении к /token, срок 120 секунд
Bearer token	Срок 3600 секунд, нужен для /userinfo

Значение SESSION_SECRET сгенерировано на серверной VM командой openssl rand. OAuth-порт не опубликован наружу: клиент обращается к нему по адресу 10.20.0.1 внутри WireGuard.

4.1. Экран входа

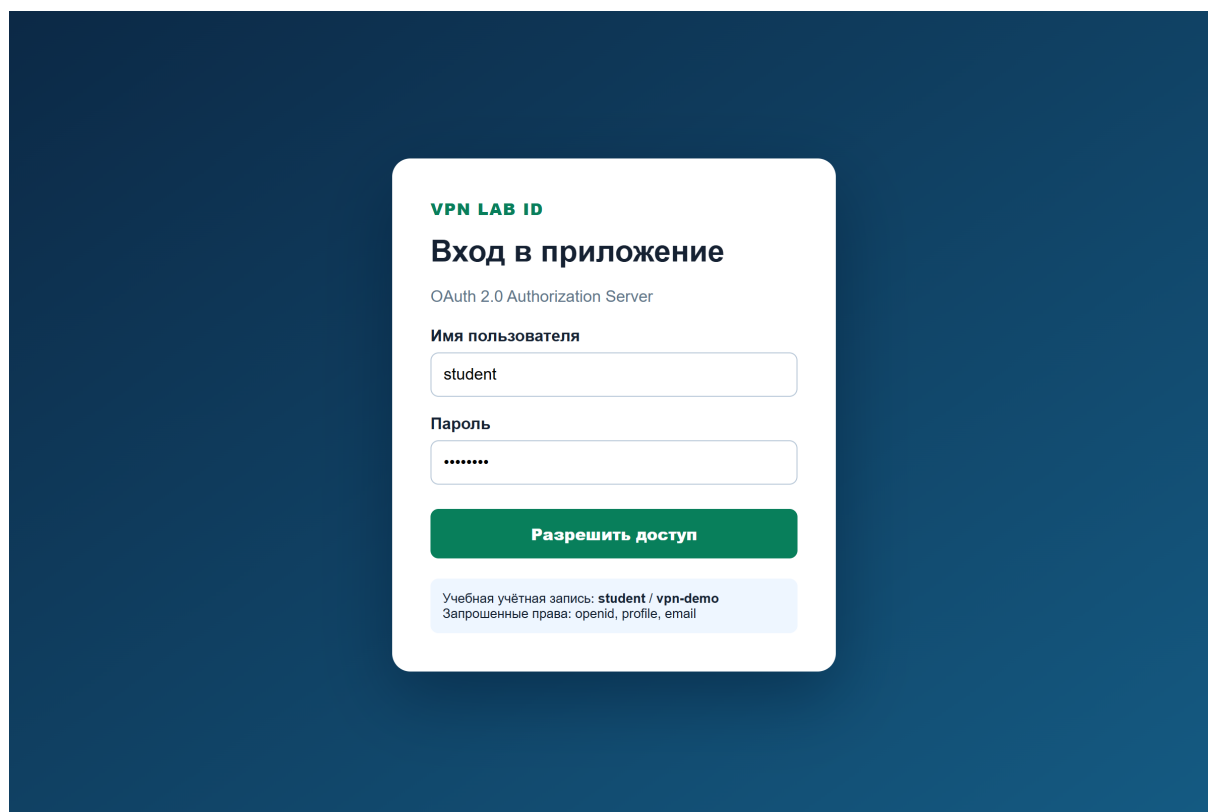


Рисунок 6 - отдельный OAuth 2.0 Authorization Server запрашивает согласие и учётные данные

Тестовая учётная запись

На учебном стенде использовались student / vpn-demo. В production пароль должен храниться как стойкий хеш и передаваться только по HTTPS.

4.2. Результат авторизации

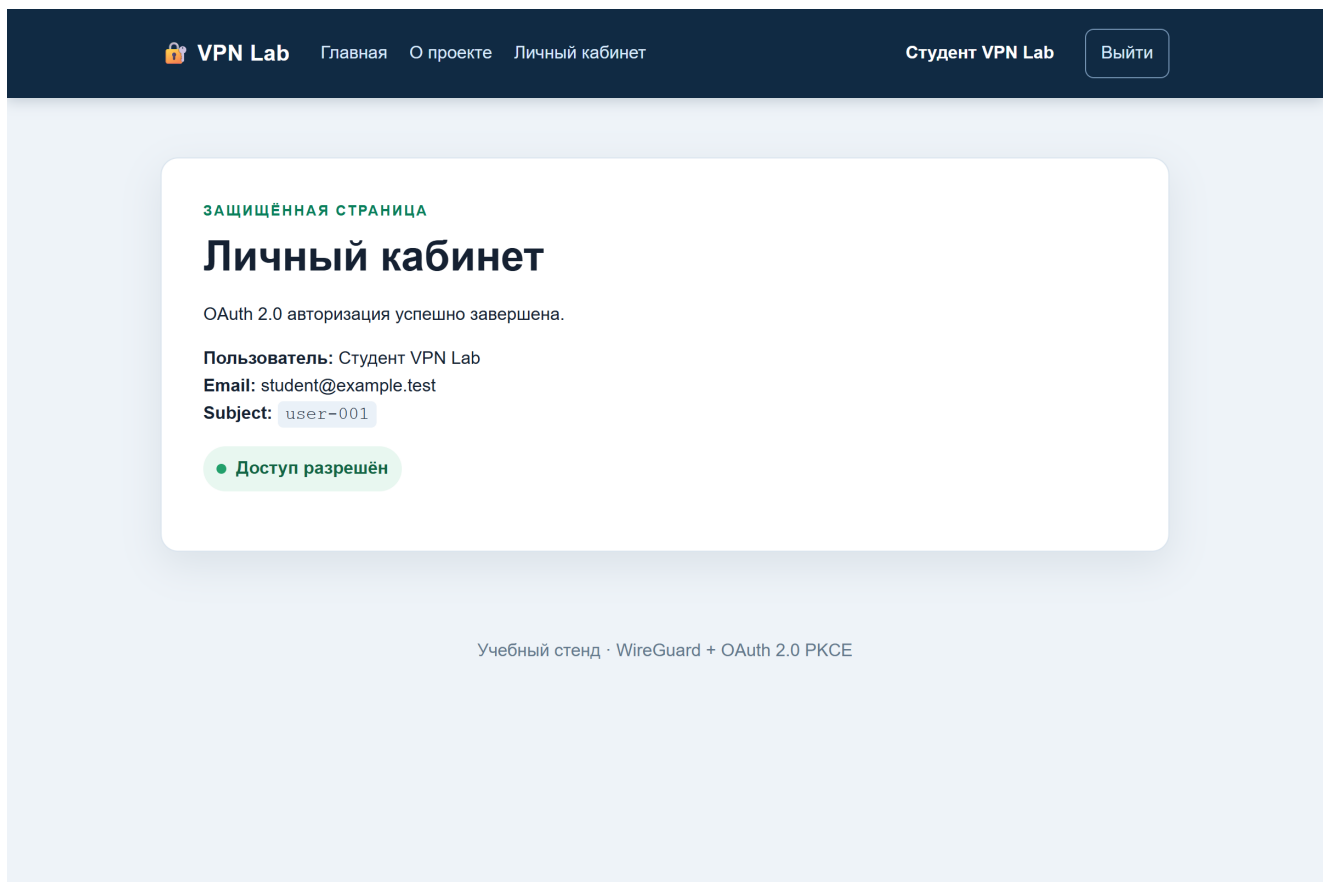


Рисунок 7 - защищённая страница после обмена authorization code на access token

Интеграционный тест автоматически прошёл всю последовательность: переход на /private, редирект к /authorize, ввод учётных данных, получение code, проверку state, обмен code + code_verifier на токен и запрос /userinfo.

```
HTTP 302 /private -> /login
HTTP 302 /login -> :9000/authorize?response_type=code&...
HTTP 302 /authorize -> /callback?code=...&state=...
HTTP 200 POST :9000/token
HTTP 200 GET :9000/userinfo
HTTP 302 /callback -> /private
HTTP 200 /private
```

4.3. Проверка OAuth с клиентской VM

Интеграционный тест запущен непосредственно на wg-client-damir при активном VPN. Скрипт создаёт HTTP-сессию, проходит обе страницы, форму /authorize, callback, обмен code + code_verifier на bearer-токен и открывает /private.

```
VM: wg-client-damir
USER: damir2

=== OAUTH 2.0 END-TO-END TEST OVER VPN ===
hostname: wg-client-damir
whoami: damir2

$ BASE_URL=http://10.20.0.1:8080 python3 integration_oauth_test.py
OK: home, about, Authorization Code + PKCE, token exchange and private page
RESULT: authorization code, PKCE, token and private page verified
```

Рисунок 8 - damir2@wg-client-damir: полный OAuth 2.0 + PKCE поток успешно выполнен через VPN

Подтверждение

На снимке видны hostname wg-client-damir, пользователь damir2, адрес серверного приложения 10.20.0.1:8080 и итог OK для home, about, token exchange и private page.

5. Выводы и сложности

Краткий анализ проделанной работы

В работе создано многостраничное веб-приложение и реализована авторизация OAuth 2.0 Authorization Code с PKCE. Стенд фактически развёрнут на двух VirtualBox VM: wg-server-damir / damir1 и wg-client-damir / damir2. WireGuard установил туннель 10.20.0.1 - 10.20.0.2; nftables и привязка служб к VPN-адресу закрыли доступ вне туннеля.

Сложности

1. Среда разработки использовала Python 3.9. Первоначальные аннотации типов нового синтаксиса были заменены на совместимые с Python 3.9 конструкции.
2. В реестре VirtualBox оставались ссылки на пустые конфигурационные файлы; рабочие VM были повторно зарегистрированы из сохранённых ready-vm's без удаления дисков.
3. Для доказательства сетевого ограничения пришлось сравнить два состояния одного клиента: wg0 down и wg0 up.
4. OAuth и веб-сервис работают по HTTP внутри VPN. Для production стоит добавить внутренний TLS-сертификат или reverse proxy.

Что можно улучшить

- Перенести OAuth-провайдер на Keycloak или внешний OIDC-сервис.
- Хранить пользователей и токены в базе данных; добавить отзыв токенов.
- Включить HTTPS внутри туннеля и Secure-cookie.
- Добавить второй VPN-клиент, мониторинг и журналирование.
- Автоматизировать повторную проверку после каждой перезагрузки VM.

Итог

Все три ключевых критерия подтверждены: многостраничное веб-приложение работает; WireGuard запущен на двух VM и ограничивает доступ; OAuth 2.0 + PKCE полностью проверен с клиентской VM.

Приложение А. Команды развёртывания, запуска и тестирования

Репозиторий и README

Исходный код: <https://github.com/AbyanovDamir/vpn-project-github-with-vm-commands>

Пошаговая инструкция: [README.md](#)

В командах ниже заменяются только значения публичных ключей и учебного пароля. Команды рассчитаны на Ubuntu 24.04.

А.1. Подготовка обеих VM

Сначала выполнить на wg-server-damir под пользователем damir1, затем повторить на wg-client-damir под пользователем damir2:

```
hostname
whoami
sudo apt update
sudo apt install -y git
git clone https://github.com/AbyanovDamir/vpn-project-github-with-vm-commands.git
cd vpn-project-github-with-vm-commands
pwd
python3 scripts/verify_project.py
```

Ожидаемый результат

На сервере hostname должен вывести wg-server-damir, whoami - damir1. На клиенте ожидаются wg-client-damir и damir2. Статическая проверка должна завершиться сообщением ОК.

А.2. Развёртывание серверной VM

Выполнить только на damir1@wg-server-damir. Пароль в примере необходимо заменить своим сложным паролем:

```
cd vpn-project-github-with-vm-commands
sudo DEMO_USER='student' \
DEMO_PASSWORD='Замените-на-сложный-пароль' \
bash scripts/install_server.sh

# сохранить выведенный публичный ключ сервера
sudo cat /etc/vpn-lab-keys/server.pub
```

A.3. Развёртывание клиентской VM

Выполнить только на `damir2@wg-client-damir`. Подставить публичный ключ, полученный на сервере:

```
cd vpn-project-github-with-vm-commands
sudo SERVER_PUBLIC_KEY='<SERVER_PUBLIC_KEY>' \
SERVER_ENDPOINT='192.168.100.10:51820' \
bash scripts/install_client.sh

# сохранить выведенный публичный ключ клиента
sudo cat /etc/vpn-lab-keys/client.pub
```

A.4. Добавление клиента на сервер

Вернуться на `damir1@wg-server-damir` и подставить публичный ключ клиента:

```
cd vpn-project-github-with-vm-commands
sudo CLIENT_PUBLIC_KEY='<CLIENT_PUBLIC_KEY>' \
bash scripts/add_client.sh

sudo wg show
bash scripts/status.sh server
```

Затем снова выполнить на `damir2@wg-client-damir`:

```
sudo systemctl restart wg-quick@wg0
bash scripts/status.sh client
ping -c 3 10.20.0.1
```

Признак исправной связи

Команда `wg show` должна содержать `latest handshake`. Ping к `10.20.0.1` должен проходить без потерь.

A.5. Запуск и перезапуск

После установки службы включены в автозапуск. Для ручного перезапуска всех компонентов использовать следующие команды.

На damir1@wg-server-damir:

```
cd vpn-project-github-with-vm-commands
sudo bash scripts/restart.sh server

# альтернативная проверка отдельных служб
sudo systemctl restart wg-quick@wg0 nftables oauth-server vpn-web
sudo systemctl is-active wg-quick@wg0 nftables oauth-server vpn-web
```

На damir2@wg-client-damir:

```
cd vpn-project-github-with-vm-commands
sudo bash scripts/restart.sh client

# альтернативная команда
sudo systemctl restart wg-quick@wg0
sudo systemctl is-active wg-quick@wg0
```

A.6. Автоматическое тестирование

Выполнить на damir2@wg-client-damir. Указать те же логин и пароль, которые были заданы при установке сервера:

```
cd vpn-project-github-with-vm-commands
DEMO_USER='student' \
DEMO_PASSWORD='Заданный-на-сервере-пароль' \
SERVER_LAN_URL='http://192.168.100.10:8080' \
bash scripts/test_client.sh
```

Успешный тест

Скрипт проверяет недоступность приложения без VPN, включает туннель, проверяет страницы и выполняет OAuth 2.0 Authorization Code + PKCE. Итоговая строка: «Все клиентские проверки пройдены».

А.7. Проверка ограничения VPN

Выполнить на damir2@wg-client-damir:

```
# 1. Выключить VPN: оба запроса должны завершиться ошибкой
sudo systemctl stop wg-quick@wg0
curl --connect-timeout 3 http://10.20.0.1:8080/health
curl --connect-timeout 3 http://192.168.100.10:8080/health

# 2. Включить VPN: запрос должен вернуть {"status":"ok"}
sudo systemctl start wg-quick@wg0
ping -c 2 10.20.0.1
curl http://10.20.0.1:8080/health
```

А.8. Ручное тестирование страниц и OAuth

При включённом VPN открыть на клиентской VM адрес <http://10.20.0.1:8080>. Проверить главную страницу, /about и кнопку входа. После авторизации должен открыться /private.

```
curl -i http://10.20.0.1:8080/
curl -i http://10.20.0.1:8080/about
curl -i http://10.20.0.1:8080/health
```

```
DEMO_USER='student' \
DEMO_PASSWORD='Заданный-на-сервере-пароль' \
BASE_URL='http://10.20.0.1:8080' \
OAUTH_URL='http://10.20.0.1:9000' \
python3 scripts/integration_oauth_test.py
```

А.9. Диагностика при ошибке

```
# сервер
sudo wg show
sudo nft list ruleset
sudo journalctl -u wg-quick@wg0 -u nftables \
-u oauth-server -u vpn-web -n 100 --no-pager

# клиент
ip -brief address show wg0
sudo wg show
curl -v --connect-timeout 3 http://10.20.0.1:8080/health
```

Важно

Приватные ключи из /etc/wireguard и /etc/vpn-lab-keys, файл /etc/vpn-lab.env и реальный пароль нельзя загружать в GitHub.